

High Performance Programming [COMTEK3, & ESD3]
Lecture 3: Parallel Program Paradigm and Design Process **02/2025**

Instruction: *The exercise problems are designed to aid your understanding of the concepts covered during the lecture. They are intended to be discussed and solved in a group of 3 – 6 students. The most important part of the exercise is the **learning process**. It is strongly recommended that you avoid the usage of LLMs such as chatGPT and similar AI models to generate solutions **without trying to solve the problems first**.*

Exercise 1: Parallelizability of Problems

Insert the following problems into the correct segment of the Venn diagram:

- (a) Square of entries of a large array
- (b) Binary search
- (c) Counting sort of a large array A
- (d) Prefix scan of an integer array
- (e) Querying a database for an entry
- (f) Computing the factorial of a large integer
- (g) Matrix-vector multiplication
- (h) Shortest path problem
- (i) Finding Fourier transform of a signal
- (j) Summing the ages of students in a class

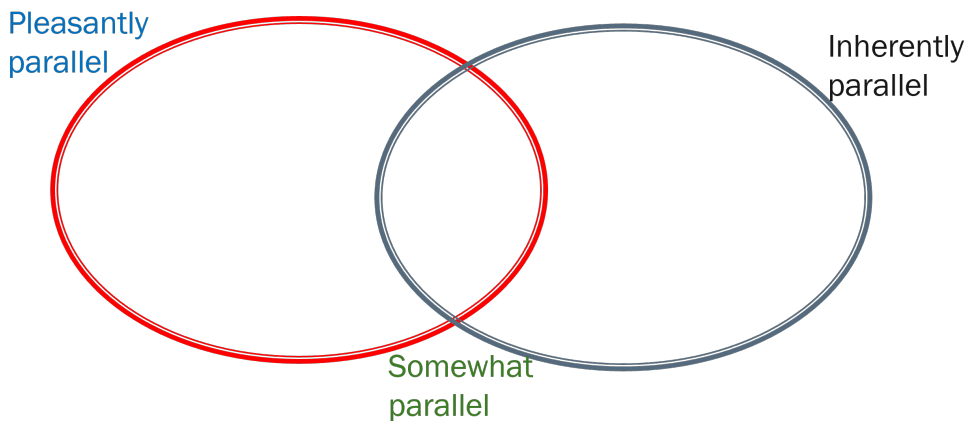


Figure 1:

Exercise 2: Task Execution Characteristics

Consider the task execution graph in Figure 2:

- (a) What are the critical path lengths for the two task dependency graphs?
- (b) If each task takes 10 time units, what is the shortest parallel execution time for each decomposition?

- (c) How many processors are needed in each case to achieve this minimum parallel execution time?
- (d) What is the maximum degree of concurrency?

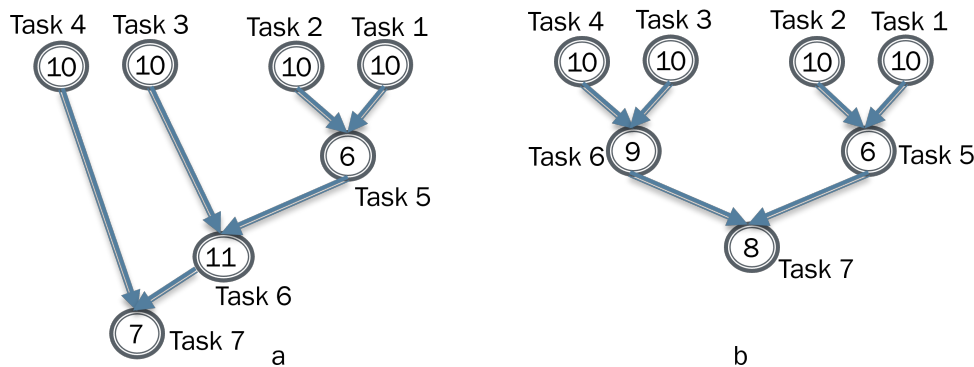


Figure 2:

Exercise 3: Task Dependency Graph

For the task graphs given in Figure 3, determine the following:

- (a) Maximum degree of concurrency.
- (b) Critical path length.
- (c) Maximum achievable speedup over one process assuming that an arbitrarily large number of processes is available.
- (d) The minimum number of processes needed to obtain the maximum possible speedup.
- (e) The maximum achievable speedup if the number of processes is limited to:
 1. 2
 2. 3
 3. 4
 4. 8

Exercise 4: Decomposition and Mapping

Consider the following sequential algorithm which is to be parallelized.

- (a) Assume a decomposition such that each execution of Line 5 is a task. Draw a task-dependency graph and a task-interaction graph.
- (b) If $n = 16$, devise a good mapping for 16 processes.
- (c) If $n = 16$, devise a good mapping for 8 processes.
- (d) Repeat Problems 4a - 4c if the statement of Line 1 in the Algorithm is changed.

Note: The XOR operation is performed using the binary representation of the indices.

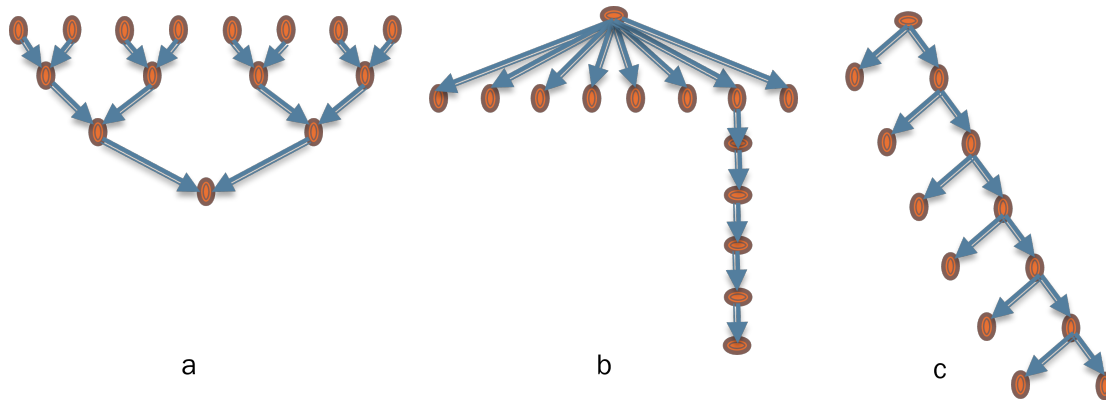


Figure 3:

Algorithm 2

```

1:  $m \leftarrow \log_2(n)$ 
2: For  $j = 1 : m$  do
3:    $k = 2^j$ 
4:   For  $i = 1 : n$  do
5:      $A[i] = A[i] + A[i \text{ XOR } 2^j]$ 
6:   End For
7: End For

```

Exercise 5: Parallel Program Design

Given an array with entries $A[i]$, consider the following recursive approach to find the partial sums:

$$S[i] = \sum_{j=0}^i A[j]$$

where:

- For $i = 0$: $S[0] = A[0]$
- For $i > 0$: $S[i] = S[i - 1] + A[i]$

- (a) Develop a parallel algorithm to find the partial sums of an array using the above technique.
- (b) Draw the binary tree representation of the algorithm in (a) for $n = 16$ and calculate:
 1. Work
 2. Speedup
 3. Efficiency

RAD